

CS135

Designing Functional Programming

Qinghao Hu

September 15, 2026

Contents

1	Admin	3
1.1	General Information	3
1.1.1	Website	3
1.1.2	Assignments	3
2	Basic	4
2.1	Values and Expressions	4
2.1.1	Beginning Student source files	4
2.1.2	Evaluating an expression	5
2.1.3	Basic arithmetic operators	5
2.2	Exact numbers	5
2.2.1	Integer division and type relationships	6
2.2.2	Other numerical operators	6
2.2.3	Real numbers	6
2.3	Constant definitions	7
2.4	Boolean expressions	7
2.4.1	Combining Boolean values	7
2.4.2	Short-circuit evaluation	8
2.4.3	De Morgan's laws	8
2.5	Lecture 01 summary	8
2.5.1	Allowed constructs	9

1 Admin

1.1 General Information

1.1.1 Website

Main website: <https://student.cs.uwaterloo.ca/~cs135>

1.1.2 Assignments

There are ten assignments, typically due Tuesday at 9:00 pm.

Computer Labs: MC2062, 2063, 3005, 3027.

Submission: On the course website. More in A00. Submit early; submit often.

2 Basic

2.1 Values and Expressions

In ordinary mathematics, we usually write operators between their operands: $2 + 3$, $6/2$, or x^2 . This is called *infix notation*. Racket instead uses a uniform notation for arithmetic and function application.

2.1.1 Beginning Student source files

In DrRacket, select the *Beginning Student* language. When writing a Racket source file, the corresponding language declaration is written at the beginning of the file:

```
#lang htdp/bsl
```

The declaration chooses the language; it is not an expression that evaluates to a number or Boolean value. The expressions and definitions come after it:

```
#lang htdp/bsl

(define x 7)
(define y 11)
(+ x y)      ; evaluates to 18
```

The line beginning with a semicolon is a comment. In the examples below, the language declaration is omitted when only an individual expression is shown.

Definition 2.1 (Prefix notation). A Racket expression is fully parenthesized, and its operator is written first:

```
(+ 2 3)
(/ 6 2)
(sqr x)
(expt x 2)
```

The expression after the operator is its argument list. The parentheses make the grouping explicit, so we do not need a separate precedence convention. The operator-first idea is also called *Polish notation*. For example, classical Polish notation writes $2 + 3$ as $+ 2 3$, without parentheses. Racket uses a parenthesized form of this idea, $(+ 2 3)$, so the parentheses explicitly show where the expression begins and ends. Thus it is helpful to think of Racket's prefix notation as parenthesized Polish notation, while using *prefix notation* as the standard CS135 term.

For example, the mathematical expressions

$$8 - 2 + 3 \quad \text{and} \quad 8 - (2 \cdot 3)$$

become, respectively,

```
(+ (- 8 2) 3)
(- 8 (* 2 3))
```

2.1.2 Evaluating an expression

Evaluation is a process of substitution. Each step replaces a subexpression by its value, and the result is still a valid (but simpler) Racket program. A substitution step chooses the leftmost, innermost subexpression with no parentheses inside it. We write the yields symbol $\Rightarrow$ between steps.

Example 2.2.

$$\begin{aligned} & (*(/62)(+12)) \\ & \Rightarrow (*3(+12)) \\ & \Rightarrow (*33) \\ & \Rightarrow 9. \end{aligned}$$

2.1.3 Basic arithmetic operators

The arithmetic operators $+$, $-$, $*$, and $/$ accept two or more values in a single step (except that $-$ also supports one argument for negation).

$$\begin{aligned} (+\ 11\ 12\ 13) &\Rightarrow 36, \\ (/ \ 720\ 5\ 4\ 3\ 2\ 1) &\Rightarrow 6, \\ (-\ 144) &\Rightarrow -144, \quad (-\ -21) \Rightarrow 21. \end{aligned}$$

In actual Racket syntax, spaces separate the arguments:

```
(+ 11 12 13)
(/ 720 5 4 3 2 1)
(- 144)
(- -21)
```

Thus, for example, $(+ \ 72)$ is an error: addition requires at least two arguments.

2.2 Exact numbers

The first number systems in CS135 are exact. Exactness means that the value is represented without rounding; for example, $3/2$ is not silently replaced by a decimal approximation.

Definition 2.3 (Natural numbers). The natural numbers are defined recursively by

$$0 \in \mathbb{N}, \quad n \in \mathbb{N} \implies n + 1 \in \mathbb{N}.$$

CS135 writes the type as `Nat` in Racket discussions. Starting at zero is especially convenient for computer science.

Definition 2.4 (Integers). The integers are

$$\mathbb{Z} = \{\dots, -3, -2, -1, 0, 1, 2, 3, \dots\}.$$

CS135 writes `Int` for this type, which extends `Nat` by adding negative numbers.

Definition 2.5 (Rational numbers). The rational numbers are numbers of the form p/q , where $p, q \in \mathbb{Z}$ and $q \neq 0$. CS135 writes **Rat**. Racket represents rational numbers exactly: `(/ 3 2)` evaluates to the exact number $3/2$. Here `(/ 3 2)` is an expression performing division, while $3/2$ is a literal way to write the resulting number.

In Racket, exact **Nat**, **Int**, and **Rat** values can be arbitrarily large or small, limited only by available memory. “Arbitrarily” does not mean infinitely large: every physical computer has finite memory.

2.2.1 Integer division and type relationships

The operators **quotient** and **remainder** perform integer division without producing a rational number:

```
(quotient 17 5) ; 3
(remainder 17 5) ; 2
```

They consume two **Ints** and produce an **Int**; applying them to rational values is an error.

The types form a subtype chain:

$$\text{Nat} \subseteq \text{Int} \subseteq \text{Rat}.$$

Thus every natural number is an integer, and every integer is rational. If an operator works for a type, it automatically works for its subtypes; an operator may still have additional restrictions, as **remainder** does. The subtype idea will recur throughout computer science.

2.2.2 Other numerical operators

The operators **max** and **min** select the largest and smallest argument:

$$(\text{max } 4 \ 8 \ -9 \ 3) \Rightarrow 8, \quad (\text{min } 4 \ 8 \ -9 \ 3) \Rightarrow -9.$$

Unlike $+$, $*$, and $/$, each also accepts a single argument:

$$(\text{max } 9) \Rightarrow 9, \quad (\text{min } 9) \Rightarrow 9.$$

Exponentiation is written with **sqr** and **expt**:

$$(\text{sqr } 12) \Rightarrow 12^2 = 144, \quad (\text{expt } 3 \ 4) \Rightarrow 3^4 = 81.$$

2.2.3 Real numbers

The rationals are enough for addition, subtraction, multiplication, division, and integer powers. But $\sqrt{2} = 2^{1/2}$ is irrational, so we need the real numbers $\mathbb{R}$. An irrational real number cannot be represented exactly in finite memory. We return to this distinction between exact and inexact numbers in Lecture 02.

2.3 Constant definitions

A mathematical variable may change as we work, but a Racket name introduced with **define** is a *constant*: it is defined once and does not change.

```
(define x 7)
(define y 11)
```

The first definition binds the value 7 to the name **x**; the second binds 11 to **y**. We can then write, for example,

```
(+ (sqr x) (* 6 x y) (sqr y) (* 9 x) (- (* 3 y)) -100)
```

A definition is evaluated by substitution. For instance,

```
(define y (+ x 4))
```

first substitutes the previously defined value of **x**, giving `(+ 7 4)`, and then evaluates that expression to 11 before binding 11 to **y**. A name must be defined before it is used, and a name cannot be redefined once it has been defined.

Meaningful constants also prevent *magic numbers*, whose purpose is otherwise hidden in a program:

```
(define lightspeed 299792458)
(define hours-in-day 24)
(define feet-in-mile 5280)
(define life-the-universe-and-everything 42)
```

2.4 Boolean expressions

In mathematics, the statement $x < 5$ describes a condition on a possibly unknown x . In Racket, a constant such as **x** already has a value, so `(< x 5)` is an expression asking whether that value is less than 5. For example:

```
(define x 4)
(< x 6) ; true
(> x 6) ; false
(= x 7) ; false
(>= 5 x) ; true
(<= 5 x) ; false
```

The comparison operators `<`, `>`, `<=`, `>=`, and `=` consume numbers and produce a Boolean value. The type is `Bool`, whose only values are `true` and `false`.

2.4.1 Combining Boolean values

The operators **and**, **or**, and **not** combine Boolean expressions:

- **and** is true exactly when all its arguments are true;
- **or** is true when at least one argument is true;
- **not** reverses a Boolean value.

Both **and** and **or** require at least two arguments, whereas **not** takes one. For example,

```
(and (or (< 2 3) (> 1 5)) (not (< 10 15)) (<= 5 5))
⇒ (and (or true (> 1 5)) (not (< 10 15)) (<= 5 5))
⇒ (and true (not true) (<= 5 5))
⇒ (and true false (<= 5 5))
⇒ false.
```

2.4.2 Short-circuit evaluation

The definitions of **and** and **or** let Racket stop early. If one argument of **and** is false, the whole expression is false; if one argument of **or** is true, the whole expression is true. This is called *short-circuit evaluation*:

```
(or (< 2 3) (> 1 5)) ⇒ (or true (> 1 5)) ⇒ true,
(and true (not true) (<= 5 5)) ⇒ (and true false (<= 5 5)) ⇒ false.
```

2.4.3 De Morgan's laws

For Boolean expressions A and B , the familiar set-theoretic identities are also valid:

$$\begin{aligned}(\text{not } (\text{and } A \ B)) &\equiv (\text{or } (\text{not } A) \ (\text{not } B)), \\ (\text{not } (\text{or } A \ B)) &\equiv (\text{and } (\text{not } A) \ (\text{not } B)).\end{aligned}$$

For example,

```
(not (or (< 2 3) (> 1 5)))
≡ (and (not (< 2 3)) (not (> 1 5)))
≡ (and (>= 2 3) (<= 1 5)).
```

2.5 Lecture 01 summary

Lecture 01 builds the first layer of a mathematical model of computation: values and expressions, then constants and Boolean expressions. The next lectures add functions, conditional expressions, and recursion; together these will form a complete small model of a computer. Lists will later make it possible to organize data and study algorithms and data structures.

The main ideas to remember are:

- write mathematical operations in fully parenthesized prefix notation;
- evaluate by valid substitution steps until a value is reached;
- distinguish exact **Nat**, **Int**, and **Rat** values;
- treat **defined** names as constants; and
- use Boolean expressions to represent true-or-false questions.

2.5.1 Allowed constructs

The Lecture 01 allowed-constructs list is:

```
( ) + - * / = < > <= >= and define expt false max min not or  
quotient remainder sqr true Bool Int Nat Rat
```

Assignments may permit the constructs from a particular lecture, together with any restrictions stated on the assignment's first page.